

LISTA DE EXERCÍCIOS 1.3

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Segundo a ANSA Reference Manual e o RM-ODP da ISO, existem diferentes tipos de transparência, referidas em conjunto como transparência de distribuição. Explique e dê exemplos destes diferentes tipos de transparência.

RESPOSTA: Explicações disponíveis nos slides 12 a 16. Transparência de acesso (NFS on boot), transparência de localização (URL sem IP), transparência de concorrência (RAID, HPC), transparência de replicação (RAID, redundância transparente de servidores web), transparência de falhas (RAID, redundância transparente de servidores web), transparência de mobilidade (sistema de telefonia celular), transparência de desempenho (ex. Regulagem de CPU em notebooks, escalonamento de processos em SDs), transparência de escalabilidade (inserção remoção de nós em HPC).

2. Como os relógios em dois computadores que estão ligados por uma rede local podem ser midsincronizados, sem fazer referência a uma fonte de tempo externa? Que fatores limitam a precisão do procedimento que você descreveu? Como os relógios poderiam ser sincronizados considerando um grande número de computadores conectados pela Internet? Discuta a precisão deste procedimento.

RESPOSTA:

- Considerando a ausência de acesso a Internet: relógios lógicos de Lamport.

- Considerando a acesso a Internet: I have 5 servers on a LAN without Internet connection. I need them to keep the clock in sync among them. I could configure them as NTP peers, and set a high stratum for the local clock on one of them. In this way, the other four would sync with that clock.

What I actually want, is them to agree on a time using all of the 5 local clocks (i.e. doing some kind of average), for reasons of robustness and precision. Is it possible with NTP?

PS: I do not want to use an external clock source. And no scripting outside NTP features, that could only make precision worse :)

→If you average 5 drifting clocks, the only thing you get is another drifting clock that's harder to correct. It won't be more precise. NTP uses multiple servers to increase precision because it takes network latency into account. Since all your systems are on a fast local network, you just need one server.

Set up two systems to be NTP server, one a primary, and if you feel the need, one a backup. Have all other systems synchronize to them. This will be significantly easier to set up than the clock-averaging solution, and you won't have to develop any crazy scripts.

→You might be able to have one of them listen for the times from each computer, perform an average, set the average as it's own time, and broadcast that time for all the other computers. It seems a little excessive, though.

→You can set up one of them as ntp server which will broadcast its time on the local network and the others as slaves to listen on the local network. Considering the average part, you can probably write a script on the local server to collect times from all the slaves get the average and update own time dleon all the servers.

→I wish I could give a definitive proposal, but I don't know enough about your environment. No matter what you'll likely be doing some sort of script kung fu. If it's unix/linux I would set everyone up with SSH authorized keys to poll each others' date +%s command (to get the epoch), average those times with awk or something, and then set the machine's own local date. Or perhaps it would be more secure (and reliable) to have one authoritative machine check everyone's time in the same manor, average it, and then provision itself and every other host to that average. On Windows you'll probably be looking into VBScript and WMI. You may run into some weird problems if anyone's clock drifts forward from the average and my guess is about half of them will ;). Future timestamps can be rather strange. It will be up to you to determine how frequently this synchronization will need to occur.

3. Por que ás vezes, é tão difícil ocultar a ocorrência e a recuperação de falhas em um sistema distribuído?

RESPOSTA: isso ocorre devido a interdependência entre infra-estruturas. Exemplo: no caso de um servidor web, é possível inserir redundância para evitar a perda de clientes por uma questão de indisponibilidade do serviço. Problema: como prever falhas de energia? Até que ponto é possível a recuperação de falhas? Redundância possui seus limites. Imagine o espelhamento de 3 servidores. Qual é a probabilidade de falha dos 3 servidores? E se o link falhar? Uso de linhas privadas para redirecionamento de tráfego? Existe a necessidade de estudo de capital a ser investido e o prejuízo gerado em caso de falhas de um sistema. Até que ponto é interessante investir para a recuperação de falhas? ROI?

4. Descreva o que quer dizer sistema escalável.

RESPOSTA: Sistema distribuído deve possuir escala adaptável quanto ao número de recursos e usuários.